

TECNICATURA UNIVERSITARIA EN CIBERSEGURIDAD

DESARROLLO DE SOFTWARE SEGURO

Primer TP obligatorio

Dante Gabriel Balbuena Atar

Profesor: Juan Pablo Villalba

Tucumán, Tafí Viejo – 09/09/2025

Contenido

Primer TP obligatorio

1. INTRODUCCIÓN

2. DESARROLLO

Identificación de Riesgos

Soluciones Propuestas

Fragmentos del Código Endurecido

3. CONCLUSIONES

4. REFERENCIAS



1. Introducción

La presente actividad se relaciona con el análisis de un código en **Flask y MySQL** utilizado en una empresa ficticia llamada *DataCorp Solutions*. El objetivo es identificar **riesgos de seguridad** en la aplicación, proponer **soluciones** y documentarlas de forma clara y sintetizada.

Este análisis se centra en revisar aspectos críticos de seguridad y aplicar prácticas recomendadas para el desarrollo seguro. Se hará referencia a guías reconocidas como **OWASP Top 10** y a conceptos de **OAuth 2.0** y **JWT**, integrando teoría con práctica.

Según *OpenAI* (2025), el uso de ChatGPT permitió elaborar fragmentos de código y consolidar buenas prácticas de seguridad.

2. Desarrollo

Identificación de Riesgos

1. **Autenticación básica (BasicAuth):** uso de credenciales fijas en el código, transmitidas en cada request sin cifrado.
2. **Credenciales expuestas:** usuario y contraseña de MySQL hardcoded en texto plano.
3. **Ausencia de validación de entradas:** campos como `user_id` o `phone_number` permiten inyección y datos maliciosos.
4. **Manejo inseguro de errores:** mensajes internos se devuelven al cliente (fuga de información).
5. **Exposición del servicio en 0.0.0.0:** sin TLS ni restricciones de acceso.
6. **Uso de fechas locales:** inconsistencia en registros, falta de zona horaria (UTC).
7. **Cierre inadecuado de conexiones:** riesgo de fugas de recursos al no usar bloques `finally`.

Soluciones Propuestas

- **Autenticación:** reemplazar BasicAuth por un esquema de **tokens Bearer**, o bien adoptar **OAuth 2.0 con JWT** para mayor robustez..
- **Gestión de secretos:** almacenar credenciales en **variables de entorno** o gestor de secretos (`.env`, `.gitignore`, Azure Key Vault, etc) permitiendo rotar contraseñas.
- **Validación de entradas:** implementar función `validate_input` con regex y listas blancas, siguiendo las recomendaciones de OWASP .
- **Manejo de errores:** respuestas genéricas con códigos HTTP adecuados y logging interno detallado.
- **Seguridad en transporte:** uso obligatorio de **HTTPS/TLS** para prevenir ataques man-in-the-middle .
- **Protecciones adicionales:** aplicar **rate limiting** para mitigar ataques DDoS .
- **Fechas y horas:** registrar siempre en **UTC** para coherencia y trazabilidad.
- **Cierre seguro:** uso de `try/finally` para cerrar conexiones y cursores.

Fragmentos del Código Endurecido

Decorador de autenticación por token:

```
@require_token

def wrapper(*args, **kwargs):

    auth = request.headers.get('Authorization', '')

    if not auth.startswith('Bearer '):

        return jsonify({'status': 'error'}), 401

    token = auth.split(' ', 1)[1].strip()

    if token not in ALLOWED_TOKENS:

        return jsonify({'status': 'error'}), 403

    return f(*args, **kwargs)
```

Validación de entradas:

```
def validate_input(data):

    if not USER_RE.match(str(data['user_id'])):

        errors.append('user_id inválido')

    if not PHONE_RE.match(str(data['phone_number'])):

        errors.append('phone_number inválido')
```

Inserción en la base de datos con parámetros seguros:

```
cursor.execute("""

    INSERT INTO user_data (user_id, phone_number, date_stamp, time_stamp, option1,
option2, full_date_time)

    VALUES (%s, %s, %s, %s, %s, %s, %s)

""", (...))
```

Nota sobre OAuth 2.0 y JWT:

En el trabajo se implementó un esquema de **tokens Bearer simples** para reemplazar la autenticación básica. Esto cumple con lo solicitado en la consigna y eleva la seguridad del código original. Sin embargo, en entornos profesionales se recomienda dar un paso más: utilizar **OAuth 2.0 como protocolo de autorización** y **JWT como formato de token**. De esta manera se obtienen beneficios adicionales como:

- Delegación de permisos sin exponer credenciales del usuario.
- Tokens con firma digital y fecha de expiración.
- Interoperabilidad con sistemas modernos y escalabilidad de la aplicación.

3. Conclusiones

El análisis me permitió comprender de manera práctica cómo pequeños descuidos en el código pueden convertirse en riesgos críticos de seguridad. Ver que la autenticación básica y las credenciales en claro representan vulnerabilidades tan serias me hizo tomar conciencia de lo fácil que es dejar puertas abiertas a ataques si no se aplican buenas prácticas desde el inicio.

Las soluciones que trabajé —como el uso de tokens Bearer, OAuth 2.0 con JWT, variables de entorno y validaciones basadas en OWASP— no solo fortalecen esta aplicación en Flask, sino que me muestran un camino claro de cómo abordar proyectos futuros con una mentalidad de “seguridad por diseño”.

Lo más valioso de este ejercicio es que pude conectar la teoría con la práctica y entender que el desarrollo seguro no es un agregado opcional, sino un requisito fundamental para entregar software confiable y de calidad.

4. Referencias

OWASP. (2023). *OWASP API Security Top 10*. Recuperado de <https://owasp.org/www-project-api-security-top-10/>

Villalba, J. P. (s. f.). *OAuth 2.0 y JWT: Una guía detallada*.

Exsystem USA. (s. f.). *Fortaleciendo la seguridad en el desarrollo de APIs: Una guía avanzada*. Recuperado de <https://www.exsystemusa.com/es/single-post/fortaleciendo-la-seguridad-en-el-desarrollo-de-apis-una-gu%C3%ADa-avanzada>

IBM. (2023). *¿Qué es la seguridad de las API?*. Recuperado de <https://www.ibm.com/es-es/topics/api-security>

OpenAI. (2025). *ChatGPT [Modelo de lenguaje grande]*. Recuperado de <https://chat.openai.com/>